

Chapitre 6

Gestion des Processus et des services

Introduction : Le changement d'échelle

Vos acquis en Programmation Système

Vous savez déjà comment un processus naît (`fork()`, `exec()`), comment il meurt (`exit()`), et comment le noyau l'ordonnance.

Le défi de l'Administration Système

En production, le défi n'est plus de créer un processus isolé, mais d'orchestrer **des centaines de services** (serveurs web, bases de données) fonctionnant simultanément sur un serveur.

Les enjeux modernes

L'administrateur système doit garantir trois piliers :

- ❶ **La résilience** : Redémarrage automatique en cas de crash.
- ❷ **L'isolation** : Empêcher un service d'épuiser la RAM ou de corrompre le disque.
- ❸ **L'observabilité** : Centraliser et analyser des millions de lignes de logs.

L'évolution du paradigme d'initialisation

Le PID 1 : La clé de voûte du système

Le tout premier processus lancé par le noyau (PID 1) est responsable de l'initialisation de l'espace utilisateur et de l'adoption des processus orphelins.

Génération	Philosophie architecturale
SysVinit (<i>Historique</i>)	Exécution séquentielle de scripts Bash. Fragile, lent, incapable de surveiller l'état réel des démons.
systemd (<i>Moderne</i>)	Gestionnaire de système complet. Démarrage parallélisé par graphe de dépendances et isolation native via les <i>Cgroups</i> du noyau.

Ce chapitre se concentre exclusivement sur les standards de l'industrie moderne propulsés par systemd.

Processus, démons et services

Définition

Un processus est une instance d'un programme en cours d'exécution. Chaque lancement d'application ou commande crée un nouveau processus.

Démon

Processus spéciaux s'exécutant en arrière-plan, fournissant des services essentiels au système ou à d'autres programmes. Lancés au démarrage et actifs jusqu'à l'arrêt.

Service

Un service désigne une application ou un ensemble de fonctionnalités exécutées en arrière-plan pour fournir des capacités spécifiques. Souvent implémenté sous forme de démon, le terme met l'accent sur la fonction offerte.

La commande ps (process status)

Syntaxe de base

```
ps [options]
```

Options courantes

- e : Affiche tous les processus.
- f : Format complet (informations détaillées).
- -u [utilisateur] : Processus d'un utilisateur spécifique.
- aux : Tous les processus avec informations détaillées.

Exemple d'utilisation

```
ps aux
```

La commande pstree (arborescence)

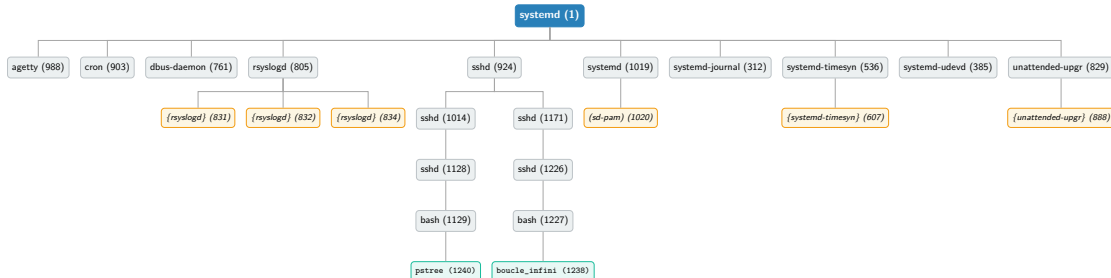
Syntaxe de base

```
pstree [options]
```

Exemple d'utilisation

```
pstree -p
```

-p : affiche les PIDs à côté de chaque processus.



Anatomie d'un Signal : Vision Noyau

Rappel de Programmation Système

Un signal n'est pas une simple "commande", c'est une interruption logicielle asynchrone (IPC) délivrée par le noyau Linux pour notifier un événement à un thread ou un processus.

Le cycle de vie d'un signal en production

- **Génération** : Par une exception matérielle, le terminal (Ctrl+C), ou un appel système (`kill()`).
- **Livraison** : Le noyau met à jour le masque des signaux en attente dans la structure du processus cible.
- **Traitement** : L'action est soit par défaut (terminaison, coredump), soit ignorée, soit interceptée via un *handler* (fonction de rappel définie par le développeur).

L'Anti-Pattern SIGKILL

La dangerosité de `kill -9 (SIGKILL)`

SIGKILL (9) et SIGSTOP (19) sont les seuls signaux **strictement non-interceptables** par le processus cible. Le noyau détruit la tâche immédiatement, sans laisser l'application exécuter son *handler*.

Conséquences dramatiques sur un serveur

- **Corruption d'I/O** : Les buffers en RAM ne sont pas vidés (*flush*) sur le disque (risque majeur de corruption pour PostgreSQL, MySQL).
- **Fuite de ressources** : Les verrous (locks) IPC ne sont pas libérés. Les sockets réseau (ports) restent bloqués dans l'état `TIME_WAIT`.
- **Filiation** : Les processus enfants ne sont pas prévenus et deviennent orphelins (rattachés au PID 1).

Règle d'or de l'administrateur : On n'utilise SIGKILL qu'en ultime recours face à un processus zombie ou bloqué en I/O.

Le *Graceful Shutdown* géré par systemd

Lors de l'arrêt d'un service (`systemctl stop`), le PID 1 n'utilise jamais SIGKILL. Il gère un mécanisme à états stricte pour permettre un arrêt propre (*Graceful Shutdown*).

Directives de gestion dans le fichier `.service`

```
[Service]
# Signal envoyé en premier pour demander un arrêt propre
KillSignal=SIGTERM

# Temps accordé au processus pour fermer ses sockets
TimeoutStopSec=30s

# Comportement du Cgroup : qui doit recevoir le signal ?
KillMode=mixed      # SIGTERM au main PID, puis SIGKILL à tous
```

Scénario typique

Le service a 30 secondes pour fermer ses connexions et vider ses buffers. Passé ce délai, systemd nettoie brutalement.

L'arrière-plan dans le terminal

Lorsqu'on lance une commande, elle bloque le terminal (Foreground).

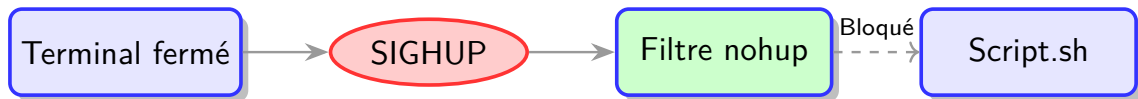
- **Lancer en arrière-plan** : Ajouter un &

```
$ ./script_long.sh &  
[1] 45678
```

- **Le problème** : Si on ferme le terminal SSH, le processus reçoit un SIGHUP (Signal Hang Up) et meurt !

Rendre un processus persistant : nohup

nohup (No Hang Up) intercepte le signal de déconnexion, permettant au script de survivre à la fermeture du terminal.



```
$ nohup ./script_long.sh &  
# Les sorties standard (stdout/stderr) sont redirigées vers nohup.out
```

Redirection personnalisée

```
nohup commande > fichier_sortie.log 2>&1 &
```

Planifier des tâches

Commande cron

Introduction à cron

Définition

Démon permettant l'exécution planifiée de tâches (cron jobs) à intervalles réguliers. Outil essentiel d'automatisation.

Structure d'une entrée crontab

```
#Minute Heure JourMois Mois JourSemaine Commande
```

```
* * * * * commande_à_exécuter
```

- ① Minute (0-59)
- ② Heure (0-23)
- ③ Jour du mois (1-31)
- ④ Mois (1-12)
- ⑤ Jour de la semaine (0-7, 0 et 7 = dimanche)

Commande **cron**

Les tâches sont listées dans des fichiers « crontabs » :

- chaque utilisateur possède son propre fichier **crontab** et peut être édité par la commande :
`crontab -e`
- il y a un fichier **crontab** système et peut être édité par la commande :
`sudo crontab -e`

Autres options de **crontab** :

- `crontab -l` : lister les tâches cron.
- `crontab -r` : supprimer toutes les tâches cron.

Commande **cron**

Exemples

Tâche planifiée tous les jours à minuit

```
0 0 * * *    root    apt update && apt full-upgrade
```

Tâche planifiée tous les 2 minutes du lundi au vendredi

```
*/2 * * * 1-5  /home/info/save.sh
```

Arrêt de la machine chaque vendredi à 19h

```
0 19 * * fri  root  shutdown -h now
```

À la dixième minute toutes les 2 heures de 0 à 20

```
10 0-20/2 * * *    /chemin/vers/script.sh
```

Bonnes pratiques et sécurité avec cron

Recommandations

- Utiliser des chemins absolus pour les scripts et commandes.
- Rediriger la sortie vers un fichier de log :

```
* * * * * /chemin/vers/script.sh >> /var/log/mon_script.log 2>&1
```
- Éviter de planifier trop de tâches simultanées.

Sécurité et dépannage

- Vérifier les logs système pour les erreurs cron.
- Utiliser sudo avec précaution dans les tâches privilégiées.
- Vérifier le service : `sudo systemctl status cron`
- Vérifier les permissions des scripts et les privilèges d'exécution.
- Valider chaque script individuellement avant la mise en production.

Gestion des services avec `systemctl`

Introduction à systemd et systemctl

systemd

Système d'initialisation et gestionnaire de services standard des distributions Linux modernes. Remplace SysV init.

Avantages

- Démarrage parallèle des services (boot plus rapide).
- Gestion automatique des dépendances.
- Surveillance et redémarrage automatique en cas de défaillance.
- Journalisation améliorée (journald).

Commandes de base de systemctl

Vérifier l'état d'un service

```
systemctl status nom_du_service
```

Démarrer / Arrêter / Redémarrer

```
sudo systemctl start nom_du_service
```

```
sudo systemctl stop nom_du_service
```

```
sudo systemctl restart nom_du_service
```

Recharger la configuration sans arrêter

```
sudo systemctl reload nom_du_service
```

Activer / Désactiver au démarrage

```
sudo systemctl enable nom_du_service
```

```
sudo systemctl disable nom_du_service
```

Gestion avancée des services avec systemctl

Lister les services

```
systemctl list-units --type=service
```

(actifs)

```
systemctl list-units --type=service --all
```

(tous)

Vérifier l'activation au boot

```
systemctl is-enabled nom_du_service
```

Masquer / Démasquer un service

```
sudo systemctl mask nom_du_service
```

```
sudo systemctl unmask nom_du_service
```

Dépendances et journaux

```
systemctl list-dependencies nom_du_service
```

```
journalctl -u nom_du_service
```

Limites structurelles de cron

Le problème architectural

Bien que standardisé (POSIX), cron est un outil hérité qui présente des limites critiques pour la gestion des infrastructures modernes :

Défauts majeurs en production

- **Aveuglement à l'état du système** : Si la machine est éteinte à l'heure prévue, l'échéance est définitivement perdue (nécessite des patches lourds comme anacron).
- **Fuite de ressources** : Les processus cron s'exécutent sans isolation stricte. Un script mal codé peut consommer 100% du CPU/RAM sans être freiné par le noyau.
- **Fragmentation des journaux** : La gestion des erreurs (stderr) est rudimentaire, nécessitant des redirections complexes sous peine de polluer le système de mails locaux (/var/mail/).
- **Absence de gestion de dépendances** : Impossible d'ordonner simplement "Lance cette tâche toutes les heures *uniquement* si le serveur de base de données est actif".

Le paradigme des systemd timers

La séparation des responsabilités

Systemd résout ce problème en séparant l'action du déclencheur. Un timer nécessite deux fichiers : un `.service` (l'action à exécuter) et un `.timer` (la planification).

Bénéfices architecturaux

- **Persistence native** (`Persistent=true`) : Si le système était hors tension, le *timer* rattrape l'exécution manquée dès le démarrage.
- **Isolation par Cgroups** : Héritant des propriétés de son `.service`, la tâche peut être drastiquement limitée en ressources (ex : `MemoryMax=500M`).
- **Centralisation des logs** : Les sorties standards sont nativement interceptées et indexées par journald (`journalctl -u mon-service`).
- **Graphe de dépendances** : Le *timer* peut exiger que `network.target` soit atteint avant de déclencher l'action.

Modifier un service existant

- Utiliser `systemctl edit --full <name>` pour personnaliser
- Ne pas modifier directement les fichiers sous `/lib/systemd/system/`
- Après modification : `sudo systemctl daemon-reload`

Dépannage au démarrage

- `journalctl -b` pour logs du boot
- `journalctl -u myservice.service` pour logs d'une unité
- Vérifier dépendances et targets

Contexte : sauvegarde automatisée

- Objectif : planifier une sauvegarde quotidienne et exposer un service de vérification
- Outils : script Bash, service systemd, crontab (ou timer)

Script : /usr/local/bin/sauvegarde.sh

```
#!/bin/bash

# Arrêt du script en cas d'erreur
set -e

# Définition des variables
DEST="/var/save/monapp"
SOURCE="srv/monapp/data"

# Création du répertoire de destination s'il n'existe pas
mkdir -p "$DEST"

# Création de la sauvegarde avec un format de date ISO (plus facile à trier)
# l'option -C permet de se déplacer dans la racine / pour l'exécution tar
# puis on donne le chemin relatif.
tar cfz "$DEST/data_$(date +%F).tar.gz" -C / "$SOURCE"

# Nettoyage des archives de plus de 7 jours
find "$DEST" -type f -mtime +7 -delete
```

Unit file : sauvegarde.service

Edition

```
sudo systemctl edit --force --full sauvegarde.service
```

Création du fichier : /etc/systemd/system/sauvegarde.service

```
[Unit]
Description=Sauvegarde quotidienne des données de monApp
After=network.target

[Service]
Type=oneshot
ExecStart=/usr/local/bin/sauvegarde.sh

[Install]
WantedBy=multi-user.target
```

Planification : timer systemd (option)

Edition

```
sudo systemctl edit --force --full sauvegarde.timer
```

Création du fichier : /etc/systemd/system/sauvegarde.timer

```
# sauvegarde.timer
[Unit]
Description=Timer pour sauvegarde quotidienne

[Timer]
OnCalendar=daily
Persistent=true

[Install]
WantedBy=timers.target
```

Activation

```
sudo systemctl enable --now sauvegarde.timer
```

- `systemctl status sauvegarde.service`
- Consulter logs : `journalctl -u sauvegarde.service`
- Vérifier présence des fichiers dans `/var/save/monapp`

- Exécuter les services avec l'utilisateur minimum nécessaire (User=)
- Surveiller l'utilisation CPU/MEM et mettre des limites (MemoryMax=)
- Utiliser les timers systemd pour cohérence et logs centralisés

Sécurisation des processus via systemd (Sandboxing)

Le principe de moindre privilège

Un service compromis (ex : faille sur un serveur web) exécuté en root donne un accès total au système. L'administrateur doit utiliser systemd pour limiter le rayon d'action du processus.

Directives d'isolation dans le fichier .service

```
[Service]
# Exécution avec des privilèges restreints
User=www-data
Group=www-data

# Isolation via les Namespaces du noyau
ProtectSystem=strict      # /usr, /boot, et /etc en lecture seule
PrivateTmp=true           # Monte un /tmp privé invisible des autres
ProtectHome=true          # Rend /home totalement inaccessible
NoNewPrivileges=true       # Interdit l'élévation via les bits SUID
```

L'isolation des processus : Les Namespaces

Définition (Le "Ce que je vois")

Apparus progressivement dans le noyau Linux, les **Namespaces** permettent de partitionner les ressources du noyau de telle sorte qu'un ensemble de processus voit un sous-ensemble isolé des ressources globales du système.

Les principaux types de Namespaces

- **MNT (Mount)** : Isole les points de montage. Le processus a sa propre arborescence de système de fichiers.
- **PID** : Isole l'arbre des processus. Le processus isolé se voit attribuer le PID 1 dans son propre namespace.
- **NET (Network)** : Isole la pile réseau (interfaces, tables de routage, règles iptables indépendantes).
- **USER** : Permet à un processus d'être root dans son namespace, mais un utilisateur standard sur le système hôte.

Le contrôle des ressources : Les Cgroups

Définition (Le "Ce que je peux consommer")

Les **Cgroups** (Control Groups) sont une fonctionnalité du noyau permettant de limiter, compter et isoler l'utilisation des ressources matérielles (CPU, mémoire, I/O disque, réseau) pour un groupe de processus.

Principes de fonctionnement

- **Hiérarchie** : Organisés sous forme d'arbre virtuel (monté sous `/sys/fs/cgroup/`).
- **Accounting** : Permet de mesurer précisément ce qu'un service consomme.
- **Limitation** : Permet d'imposer des plafonds stricts (OOM Killer déclenché uniquement dans le périmètre du cgroup en cas de dépassement).

La révolution systemd

systemd place **automatiquement** chaque service dans son propre cgroup dédié, mettant fin au chaos des processus se battant pour les ressources globales.

Application : Le Sandboxing avec systemd

Cgroups + Namespaces = Conteneurisation native

Grâce à systemd, l'administrateur n'a pas besoin de coder des appels noyau complexes. Il suffit de déclarer des directives dans le fichier `.service` pour activer ces mécanismes.

Exemple d'intégration dans un fichier Unité

```
[Service]
ExecStart=/usr/bin/mon_serveur_web

# ---- Utilisation des Cgroups ----
MemoryMax=500M           # Limite stricte de RAM (OOM intra-cgroup)
CPUWeight=50              # Priorité CPU basse par rapport au reste

# ---- Utilisation des Namespaces ----
PrivateNetwork=yes        # Crée un namespace NET (loopback uniquement)
PrivateTmp=yes            # Crée un namespace MNT (masque /tmp aux autres)
ProtectSystem=strict       # Rend /usr et /boot en lecture seule (MNT)
```

Gestion des logs

Introduction aux logs sous Ubuntu

- Les logs sont des fichiers qui enregistrent les événements du système
- Essentiels pour la maintenance et le dépannage
- Sous Ubuntu, la plupart des logs sont stockés dans `/var/log/`
- Format généralement textuel, certains peuvent être compressés

Principaux fichiers de logs

`/var/log/syslog` Événements système généraux

`/var/log/auth.log` Authentifications et autorisations

`/var/log/kern.log` Messages du noyau Linux

`/var/log/dmesg` Messages du démarrage du système

`/var/log/apache2/` Logs du serveur web Apache (si installé)

`/var/log/apt/` Logs des opérations d'installation de paquets

Outils pour consulter les logs

Commandes de base :

- `cat`
- `less`
- `tail`
- `grep`

Exemples :

- `tail -f /var/log/syslog`
`-f` (`--follow`) : Suit les ajouts au fichier en temps réel. Souvent utilisée pour surveiller les fichiers de logs. Pour arrêter la commande, utilisez `Ctrl + C`.
- `grep -i error /var/log/syslog`

- Ubuntu utilise systemd qui intègre le système de logs journald
- journalctl permet d'accéder à ces logs
- Avantages :
 - Indexation et recherche rapide
 - Filtrage par unité, priorité, date
 - Format structuré avec métadonnées
- Exemples :
 - journalctl -b (logs depuis le démarrage)
 - journalctl -p err (erreurs uniquement)

- Les logs peuvent rapidement occuper beaucoup d'espace
- Ubuntu utilise logrotate pour gérer leur cycle de vie
- Configuration dans `/etc/logrotate.d/`
- Fonctionnalités :
 - Compression des anciens logs
 - Suppression après une période définie
 - Rotation basée sur la taille ou la date
 - Exécution de scripts après rotation

Rotation des logs

Exemple du fichier `/etc/logrotate.d/rsyslog`

```
/var/log/syslog
{
    rotate 7          # Conserve les 7 derniers logs avant suppression
    daily             # Effectue une rotation des logs chaque jour
    missingok         # Ne génère pas d'erreur si un fichier log est absent
    notifempty        # Ne fait rien si un fichier log est vide
    delaycompress     # Retarde la compression du dernier fichier tourné
    compress          # Compresse les anciens fichiers log
    postrotate        # Exécute "rsyslog-rotate" rotation
        /usr/lib/rsyslog/rsyslog-rotate
    endscript        # Fin du script
}
```

Investigation avancée avec journalctl

La puissance des métadonnées

Contrairement aux fichiers textes classiques (/var/log/syslog), journald indexe les logs. Il est possible de croiser des requêtes complexes sans utiliser grep.

Cas d'usage critiques

Logs d'un exécutable spécifique (même s'il a planté)

```
journalctl _EXE=/usr/bin/sshd
```

Erreurs du noyau lors du boot précédent (très utile en cas de crash)

```
journalctl -k -b -1
```

Suivre les erreurs d'un service en temps réel

```
journalctl -u monApp.service -p err -f
```

Logs générés durant la dernière heure

```
journalctl --since "1 hour ago"
```

Ce qu'il faut retenir

L'administration des processus sous Linux a fondamentalement changé. Il ne s'agit plus de lancer des scripts en arrière-plan avec `nohup`, mais de concevoir des services robustes :

- **Centralisation** : `systemd` unifie la gestion des démons, des planifications (*timers*) et de la journalisation (*journald*).
- **Sécurité par défaut** : L'utilisation des directives de Sandboxing (`ProtectSystem`, `PrivateTmp`) est désormais un standard industriel incontournable.
- **Gestion des IPC** : L'arrêt d'un service de production exige un *Graceful Shutdown* rigoureux pour préserver l'intégrité des bases de données, rendant l'usage de `SIGKILL` proscrit en routine.

Ouverture : Vers l'infrastructure de demain

La prochaine étape logique

Dans ce chapitre, nous avons isolé les ressources d'un processus via les **Cgroups** et restreint sa vue du système via les **Namespaces** de systemd.

Namespaces + Cgroups = Conteneurisation

Préparation au cycle Master

Les concepts d'isolation et d'orchestration vus ici au niveau local sont la fondation exacte du fonctionnement de **Docker** (encapsulation) et de **Kubernetes** (orchestration distribuée sur plusieurs serveurs).

Commandes

- `man ps`
- `man kill`
- `man systemd.service`
- `man systemd.timer`
- `man journalctl`
- `man logrotate`

Pages noyau

- `man 7 signal`
- `man 7 signal-safety`
- `man 7 cgroups`
- `man 7 namespaces`